

CSE411

Project Task 7

Implementation - Part 4

Final Project Report

Project Title

Computer Vision-Based Flood Detection and Water-Level Monitoring System

Public source-code repository

github.com/MK00315/

Computer-Vision-Based-Flood-Detection-and-Water-Level-Monitoring-System

Team 20

2023BCS0235 Mahendra K

2023BCS0232 Sanin K

2023BCS0238 Bhukya Gandhi

2022BCD0001 Ankit Subhash

1. Introduction and Project Objectives

Abstract

This project implements a computer-vision prototype for detecting cars in flooded photographs and assigning a relative flood-level category to each detected vehicle crop. COCO-pretrained YOLOv8n performs car detection, while an ImageNet-initialized ResNet-50 is fine-tuned on STURM-FloodDepth vehicle crops for five-class classification. The classifier is selected using validation macro F1 and evaluated on the official 340-image test split. It achieves 74.71% accuracy and 75.73% macro F1. The integrated pipeline is demonstrated on ten full-scene images, with a baseline/exploratory setting comparison and thirteen saved interactive tests. Scene detections are assessed qualitatively because these photographs do not have verified ground-truth boxes and flood-level labels. The implemented scope is still images and relative vehicle submergence, not exact physical water-depth measurement.

Problem statement

Flood photographs can show several partly submerged vehicles under different viewing conditions. Automatically locating these vehicles and interpreting their visible submergence can support visual assessment. However, ordinary photographs lack the calibration required to infer a reliable water depth in centimetres or metres. This project therefore reports relative categories based on vehicle-centred imagery rather than physical-depth estimates.

Objectives

1. Prepare and verify the official STURM train, validation and test splits.
2. Fine-tune a five-class ResNet-50 classifier and select its checkpoint using validation macro F1.
3. Integrate pretrained car detection, padded cropping, classification and annotated output.
4. Evaluate the classifier quantitatively and inspect integrated scene behaviour separately.
5. Provide a tested Colab prototype, reproducible source code and saved experimental evidence.

Research basis and practical approach

Vehicle-based flood assessment is motivated by earlier research, including Wan et al. [1]. The available STURM package provides labelled vehicle crops, which support classification but do not constitute a complete full-scene YOLO detection-training set. The implemented approach consequently separates general car detection from flood-level crop classification. YOLOv8n [3] is reused without detector training; ResNet-50 [4] is adapted to the target labels.

Scope

The prototype processes still images and only the COCO car class. Relative classes are Level0-Level4. Video/tracking, water segmentation, metric-depth estimation and operational alert deployment are not claimed as completed features. The work is an implementation and evaluation of established methods, not a newly invented detection algorithm.

2. Dataset and Experimental Design

Classifier dataset

STURM-FloodDepth Flooded Cars provides vehicle-centred images with five relative flood-level classes. The publisher describes crops extracted using existing vehicle boxes, enlarged around the objects and upsampled before distribution [2]. This project uses those supplied crops; it does not recreate the original box annotations or train the upscaling model. Class labels are obtained from the Level0-Level4 folder names.

Class	Training	Validation	Test	Total
Level0	640	80	80	800
Level1	564	70	72	706
Level2	892	111	113	1,116
Level3	356	44	45	445
Level4	240	30	30	300
Total	2,692	335	340	3,367

Table 1. Class distribution verified against the official split lists.

Split handling and checks

train.txt, val.txt and test.txt contain relative image paths, not pixel data. The notebook verifies the expected counts, class membership, file existence and unique listed paths. There is no exact path overlap between the three splits. Of 3,779 images found in the extracted package, 412 are not listed and remain excluded from the experiment.

Some base filename groups are shared across splits: 164 train/validation, 155 train/test and 41 validation/test groups. These indicate a related-crop limitation; they are not a count of independently verified duplicate source photographs. The official image-level split is retained, and its test scores are not described as performance on entirely unseen scenes.

Training, validation and testing roles

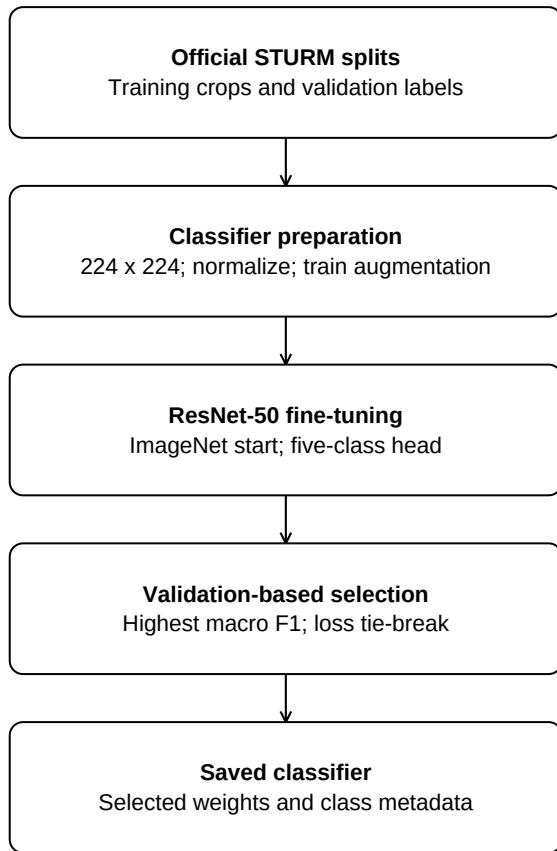
Training images update classifier weights. Validation images monitor learning and determine checkpoint selection. The held-out test crops are evaluated after selection. Task 7 reuses the trained Task 4 classifier rather than starting an additional training run.

Full-scene demonstration images

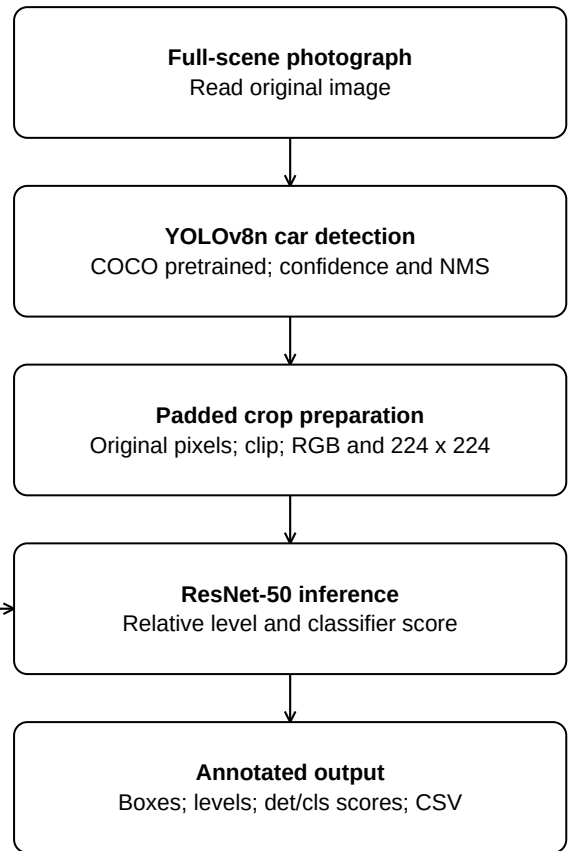
Ten separate Wikimedia Commons photographs from the earlier integrated tasks are used to exercise the detector/classifier pipeline. They are not added to classifier training and do not have verified scene-level annotations for a quantitative detection or end-to-end benchmark. Authors, links, licence identifiers and modifications are retained in image_credits.csv; selected figure credits and the complete scene list are provided in Section 12.

3. System Architecture

A. CLASSIFIER DEVELOPMENT



B. STILL-IMAGE INFERENCE



C. EVALUATION AND INTEGRATION TESTING

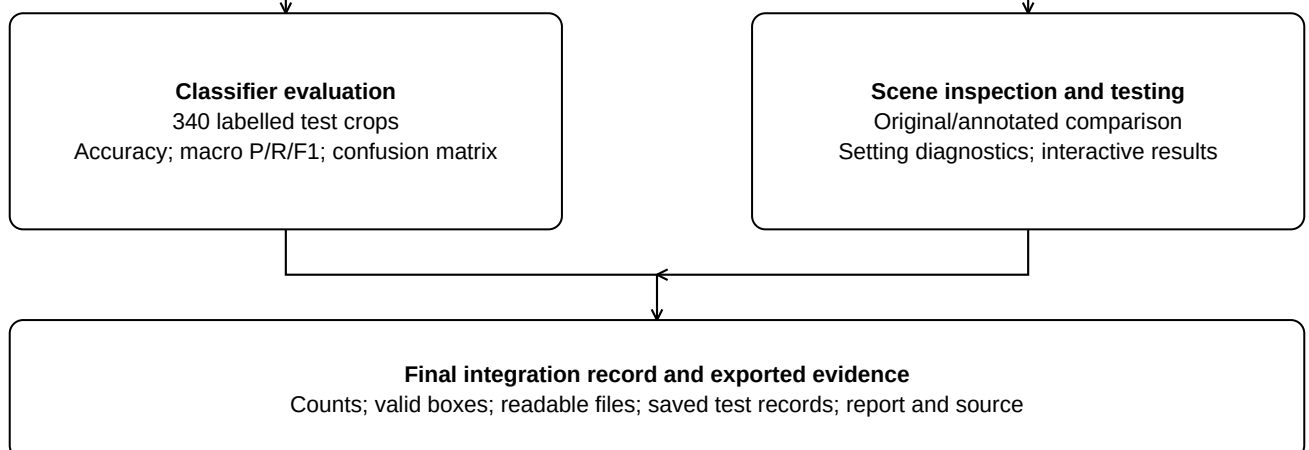


Figure 1. Implemented training, inference and evaluation workflow. The saved classifier supplies both scene inference and labelled-crop evaluation. Scene outputs feed qualitative review and integration checks.

Training augmentation is not applied at inference. The detector is not trained on STURM. The classifier test bypasses YOLO; therefore its measured accuracy is separate from complete-scene performance. Optional video processing is outside the executed final scope.

4. Methodology and Implementation

Image preparation and car detection

A full-scene image is decoded using OpenCV. COCO-pretrained YOLOv8n processes the image with car class index 2, a confidence threshold and IoU-based non-maximum suppression. Its weights are not updated. Invalid or empty boxes are rejected, and coordinates are clipped to the original image bounds.

Padded vehicle extraction

For each retained detection, the crop is expanded by 20% of its width on the left and right and 20% of its height above and below. The expanded rectangle is clipped to the image bounds. Crops are taken from the original, unannotated frame to avoid feeding drawn labels back into the classifier.

Classifier input and prediction

The crop is converted from OpenCV BGR to RGB, resized to 224 x 224, converted to a tensor and normalized using ImageNet RGB means [0.485, 0.456, 0.406] and standard deviations [0.229, 0.224, 0.225]. The saved ResNet-50 produces five logits. Softmax scores are calculated and the maximum-scoring Level0-Level4 class is selected.

Confidence handling and output records

Detector and classifier scores are displayed separately. A classifier score below 0.50 is flagged Uncertain (predicted level), not discarded or treated as a sixth class. Green/orange annotations represent the classifier flag, not a safety assessment. CSV records preserve the car box, expanded crop box, class, both scores and uncertainty status.

Setting	Baseline	Exploratory
Detector confidence threshold	0.30	0.15
Detector inference target size	640	1280
IoU threshold	0.45	0.45
Classifier uncertainty threshold	0.50	0.50
Crop padding / classifier input	20% per side / 224 x 224	Same
Detector and classifier weights	Fixed selected weights	Same

Table 2. Detector profiles. Two settings change together; this is an exploratory diagnostic comparison, not a controlled demonstration of an accuracy improvement. The baseline remains the default.

Execution environment

The supplied final notebook executed on a Colab CUDA GPU with PyTorch 2.11.0+cu130 and Ultralytics 8.4.162. PyTorch/Torchvision provide the classifier and transformations; OpenCV/Pillow handle images; pandas/NumPy handle records; scikit-learn calculates classifier metrics; Matplotlib/seaborn plot results; ipywidgets provides the interactive panel.

5. Classifier Training and Model Selection

The Task 4 training procedure starts from ImageNet-pretrained ResNet-50 and replaces its final layer with five outputs. All parameters are fine-tuned. Training uses only the official training split; validation is performed without gradient calculation. Task 7 loads the selected weights and matching history for final evaluation.

Training item	Implemented configuration
Duration / batch / seed	5 epochs / 32 images / 42
Optimizer	AdamW; learning rate 0.0001; weight decay 0.0001
Loss / gradient clipping	Inverse-frequency class-weighted cross-entropy / norm 5.0
Scheduler	ReduceLROnPlateau on validation macro F1; factor 0.3; patience 1
Checkpoint rule	Maximum validation macro F1; validation loss breaks ties

Training-only augmentation includes a 0.5-probability horizontal flip, rotation up to 8 degrees, translation up to 5%, scale 0.95-1.05, and brightness/contrast/saturation jitter of 0.15. Validation and test crops are resized and normalized without random augmentation.

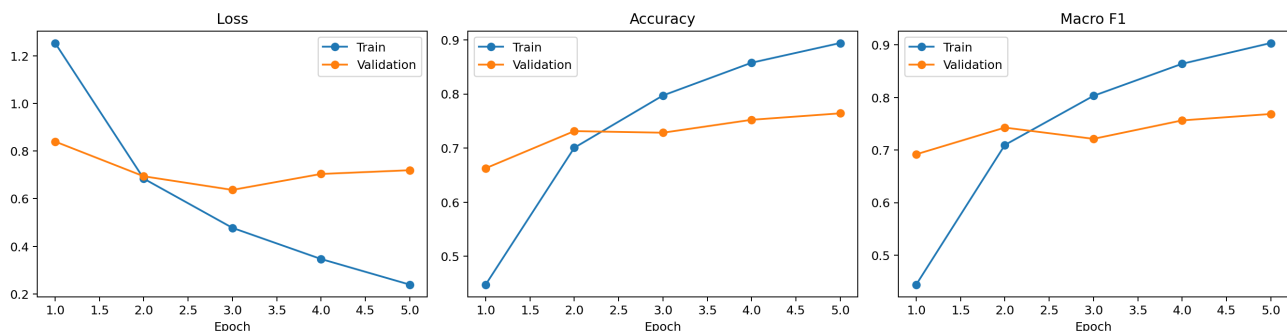


Figure 2. Original saved five-epoch training curves. These curves are inherited from the approved classifier training run, not a new Task 7 training experiment.

Epoch	Training loss	Validation loss	Validation macro F1
1	1.2534	0.8397	69.20%
2	0.6839	0.6939	74.28%
3	0.4775	0.6367	72.13%
4	0.3466	0.7036	75.64%
5	0.2392	0.7193	76.86%

Table 3. Checkpoint selection history. Epoch 5 has the highest validation macro F1 (76.86%); epoch 3 has the lowest validation loss. The chosen rule selects epoch 5. Training/validation separation and the loss increase after epoch 3 suggest a generalization limitation; five epochs are not claimed to be an optimal training duration.

6. Quantitative Classifier Evaluation

The selected classifier is evaluated on 340 labelled STURM test crops. It correctly classifies 254 images. All metrics below concern this crop-classification task and bypass YOLO detection. No scene-level mAP, detector precision/recall or end-to-end accuracy is reported.

Metric	Test result
Accuracy	74.71% (254 / 340)
Macro precision	75.39%
Macro recall	76.26%
Macro F1	75.73%

Accuracy is the fraction of correctly classified crops. Precision measures how often predictions of a class are correct; recall measures how many true examples of that class are recovered. F1 balances precision and recall. Macro scores average the five class scores equally, preventing the largest class from dominating the summary [6].

Class	Precision	Recall	F1	Support
Level0	84.62%	82.50%	83.54%	80
Level1	58.67%	61.11%	59.86%	72
Level2	75.68%	74.34%	75.00%	113
Level3	78.57%	73.33%	75.86%	45
Level4	79.41%	90.00%	84.38%	30

Table 4. Class-wise test scores. Support is the number of labelled test crops in that class.

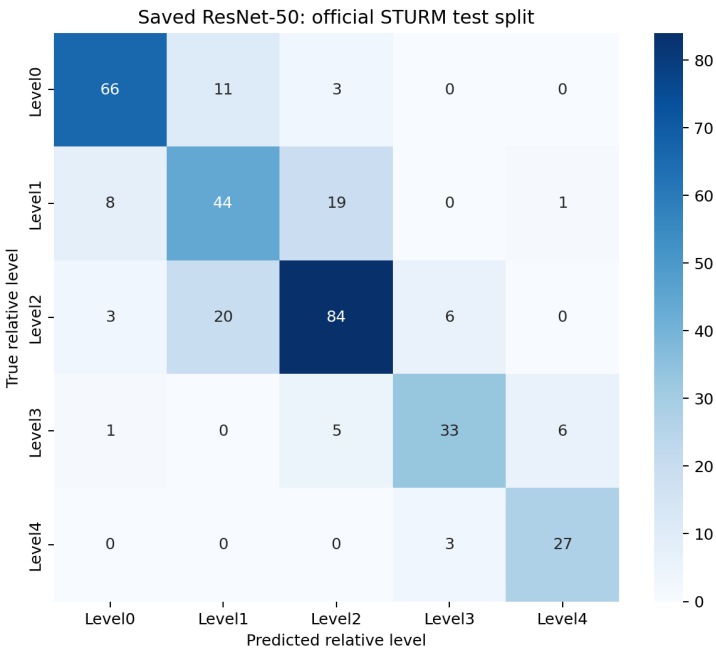


Figure 3. Original test confusion matrix. Rows are true labels; columns are predicted labels. Diagonal counts are correct predictions.

Level1 has the lowest F1 (59.86%). The confusion matrix includes 19 true Level1 crops predicted as Level2 and 20 true Level2 crops predicted as Level1. Level4 recall is 90.00%, but its support is only 30 images. These findings identify class-specific behaviour, not a guarantee for new photographs.

7. Full-Scene Integration Tests

The same detector and classifier process ten full-scene photographs under baseline and exploratory profiles. Each profile writes annotated images and per-detection records. The saved baseline has 24 boxes; the exploratory profile has 40. Both have three uncertain classifications. Baseline zero-detection scenes are 001, 003 and 008; exploratory zero-detection scene is 003.

Scene	Baseline boxes	Exploratory boxes
001 - Atcham flood road	0	1
002 - Trebic flooded street	1	2
003 - Oklahoma red car	0	0
004 - Oklahoma neighbourhood	2	5
005 - Flooded street and vehicles	4	7
006 - Cedar Rapids, 13th Avenue	2	1
007 - West Point flood damage	4	3
008 - Cedar Rapids, sunken cars	0	2
009 - Berry Street	5	6
010 - Dubai flood	6	13
Total	24	40

Table 5. Diagnostic box counts. Seven scenes have more boxes, two have fewer and one is unchanged. These are prediction counts, not counts of confirmed cars or correct classifications.



Figure 4. Scene 004: baseline (left, two boxes) and exploratory (right, five boxes). Photograph: Marvin Nauman/FEMA, public domain in the US. Prediction overlays added; both views resized for the report.

The additional boxes in Figure 4 show a change in detector behaviour. Lower confidence and larger inference size change simultaneously, so their effects cannot be isolated. Without labelled ground truth, more boxes cannot be interpreted as improved precision, recall or flood-level accuracy.

8. Interactive Testing and Visual Findings

Thirteen interactive tests were saved as an annotated JPG, detection CSV and settings JSON per test. The following table records every saved test. All use detector size 1280, IoU 0.45, classifier uncertainty threshold 0.50 and the same model weights.

Test	Scene	Detector threshold	Saved boxes
1	008	0.15	2
2	008	0.70	0
3	008	0.45	1
4	008	0.10	2
5	006	0.10	1
6	006	0.30	1
7	006	0.60	1
8	003	0.60	0
9	003	0.35	0
10	003	0.05	1
11	005	0.05	12
12	005	0.30	4
13	005	0.50	2

Table 6. Saved interactive test settings and counts, in timestamp order. Test 10 detects a candidate box in scene 003 at confidence 0.05; the baseline and exploratory batch profiles both returned zero.



Figure 5. Scene 003 original (left) and low-threshold interactive output (right, one box). Photograph: Marvin Nauman/FEMA, public domain in the US. The right view is annotated; views are resized.

A very low threshold can recover a candidate detection but can also introduce incorrect boxes. This test confirms that the interactive controls run through the integrated pipeline; it does not establish reliable recovery of all submerged vehicles. In scene 008, the exploratory output contains two partial boxes around the foreground pickup rather than two independently confirmed cars. Some clearly visible vehicles remain unboxed.

9. Validation, Limitations and Conclusion

Integration and file-consistency checks

The executed final notebook records 21 completed code cells with no saved cell-error outputs. Its integration summary reports Passed for 340 test crops, ten scenes, 24 baseline boxes, 40 exploratory boxes and thirteen interactive tests. Checks cover record/image counts, valid coordinates, readable output files and consistency between the saved records and run metadata. These checks establish technical integration, not prediction correctness.

Observed limitations

1. Detector domain mismatch: COCO-pretrained car detection can miss heavily submerged, occluded, small or unusual-looking vehicles. A missed car never reaches the classifier.
2. Crop domain mismatch: supplied STURM crops are curated/upscaled; YOLO crops can contain partial vehicles, water, background or multiple objects.
3. Class ambiguity: adjacent flood levels can look similar, and the test confusion matrix shows substantial Level1/Level2 confusion.
4. Split independence: related crop filename groups appear across official splits, limiting claims about unseen source scenes.
5. Score interpretation: softmax and detector scores are not calibrated probabilities of flood safety; a confident result can still be wrong.
6. Scene evaluation: the ten demonstrations lack verified ground-truth boxes and class labels, so no full-system accuracy, mAP, precision or recall is inferred.

Course relevance and practical contribution

The project directly applies CSE411 recognition topics: object detection, supervised classification, feature learning and neural networks. Bounding-box cropping and image transformations connect to image representation and geometric preprocessing. CNN filters learn features, but the project does not claim to implement every syllabus method such as Hough transforms, SIFT or optical flow. The practical contribution is the integrated detection-to-classification prototype, uncertainty display and explicit separation of classifier metrics from scene diagnostics.

Future improvements

Collect annotated full-scene flood photographs with car boxes and relative labels, use scene-group-disjoint splits, fine-tune a detector for submerged vehicles and test crop-domain robustness. Further work may examine class-balanced/ordinal learning, calibrated uncertainty and one-factor detector comparisons. Video frame processing with tracking and temporal consistency is an optional extension, not part of the current executed demonstration.

Conclusion

The core still-image methodology is implemented and integrated: pretrained car detection, padded cropping, five-class flood-level classification, annotated records, quantitative classifier evaluation and interactive testing. The selected classifier reaches 74.71% accuracy and 75.73% macro F1 on the official STURM test crops. Scene tests demonstrate a working prototype while revealing missed vehicles and ambiguous boxes. The system remains a relative visual-assessment prototype and does not measure exact depth or certify safe passage.

10. Source Code and Reproducibility

Public GitHub repository

github.com/MK00315/

[Computer-Vision-Based-Flood-Detection-and-Water-Level-Monitoring-System](#)

The public repository contains clean classifier-training, integrated-evaluation and Gradio notebooks, matching Python exports, dependencies, setup instructions, selected result records and this revised report. The repository is source hosting, not a permanently running demo.

Final deliverables

Deliverable	Contents
Final project report	Full methodology, architecture, experimental results, limitations and references.
Executed Task 7 notebook	The user-supplied final notebook, preserved unchanged with outputs.
Final source-code package	Clean training, evaluation and Gradio notebooks; matching Python exports; dependencies, setup instructions and licences.
Results archive	Original images, baseline/exploratory annotations, metrics, curves, credits, thirteen interactive tests and integration summary.
Gradio presentation demo	Redesigned image interface, clean source, supplied demo notebook and saved session results.

Expected Drive inputs

Use the existing CSE411_Flood_Project folder in MyDrive. Dataset input is STURM-FloodDepth with train.txt, val.txt, test.txt and flooded_cars/upscaled_images/Level0-Level4. The approved classifier is task4_outputs/resnet50_sturm_best_macro_f1.pth, with its matching resnet50_macro_f1_training_history.csv. Full-scene inputs are in task5_scene_images. The source package README provides the precise folder layout.

Running the submitted source

1. Open the clean main notebook in Google Colab and select a GPU runtime.
2. Mount Drive and confirm the dataset, checkpoint, matching history and scene paths.
3. Run the cells in order for split verification, model loading, test evaluation, scene processing and profile comparison.
4. Use the interactive panel if additional examples are needed, then rerun final checks and packaging.
5. Keep photograph credits and licence conditions with any reused image output.

The source package does not duplicate the dataset or trained weights. Existing Drive inputs are required; YOLOv8n pretrained weights download automatically. Optional classifier-training reproduction writes to a separate folder and must not be confused with the evaluated checkpoint. New training runs may give different scores across software/hardware.

Task 7 requirement check

Task requirement	Evidence supplied
Project title, team and members	Cover and notebook identity cells.
Complete proposed core methodology	Sections 2-5 and executed integrated pipeline.
Integrate and test modules	Classifier test, ten scenes, profile comparison, thirteen interactive tests and integration_checks.json.
Final report and source code	This report and the separate final source-code package.

The retained project boundary is still-image relative flood-level assessment. Video was optional in the approved scope and is not represented as completed.

11. Gradio Presentation Demo

The password-protected Gradio page wraps the completed pipeline without retraining either model. It shows the input and annotated scene together. Predictions, padded crops, downloads and project details occupy separate tabs; advanced detector controls are collapsed by default.

Saved demonstration runs

The Drive download contains two backups; the earlier four runs recur in the later ten-run snapshot. Counting each saved analysis once gives ten analyses of four photographs, thirteen prediction rows and two uncertain rows. These are not ten independent test scenes or an accuracy measurement. Trebic was processed twice with identical settings.

Run	Photograph	Det. threshold	Input size	Boxes	Uncertain
1	Cedar Rapids	0.30	640	0	0
2	Cedar Rapids	0.15	640	0	0
3	Cedar Rapids	0.05	640	0	0
4	Cedar Rapids	0.05	1280	3	0
5	Flooded street	0.30	640	4	0
6	Trebic	0.30	640	1	1
7	Trebic (repeat)	0.30	640	1	1
8	Atcham	0.30	640	0	0
9	Atcham	0.05	640	1	0
10	Atcham	0.05	1280	3	0

Table 7. Genuine saved Gradio analyses. The same checkpoint and preprocessing are used in every row; changing detector settings changes candidate detections, not the trained classifier.

Output inspection and practical use

The flooded-street example returns four boxes at the baseline settings. The Trebic example returns one uncertain classification while other visible vehicles remain undetected. Atcham returns zero boxes at baseline and more candidate boxes at lower thresholds. These outputs illustrate both a usable demonstration and the detector limitations; zero detections do not establish the absence of cars or flooding.

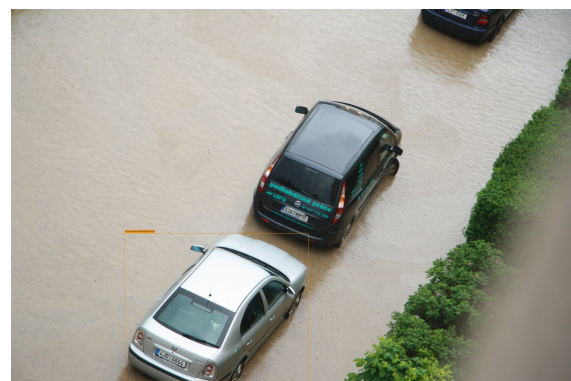


Figure 6. Saved Gradio results: flooded street (left; MarkBuckawicki, CC0) and Trebic (right; Jiri Sedlacek, CC BY-SA 4.0). Original prediction overlays are preserved; inspection copies are resized for this report. Source pages are included in the image-credit records.

Use the demo notebook in Colab with the saved classifier checkpoint. Run Sections 1-8 in order and use the printed temporary link and login. Download results or make a session backup before the runtime ends. Keep the launch-cell password output out of any public notebook. GitHub does not keep the Colab runtime or Gradio link online.

12. References and Image Attribution

- [1] Wan et al. (2024). Automatic detection of urban flood level with YOLOv8 using flooded vehicle dataset. Journal of Hydrology, 639, 131625. [Source](#).
- [2] Notarangelo, N. (2025). STURM-FloodDepth: Flooded Cars dataset. Zenodo record 14833532. [Source](#).
- [3] Ultralytics. YOLOv8 model documentation. [Source](#).
- [4] He, K., Zhang, X., Ren, S. and Sun, J. Deep Residual Learning for Image Recognition. [Source](#).
- [5] PyTorch/Torchvision. ResNet-50 model and pretrained weights documentation. [Source](#).
- [6] scikit-learn. Model evaluation and classification metrics documentation. [Source](#).
- [7] Gradio. Sharing applications and temporary demo links. [Source](#).

Scene photographs and licences

The following identifiers match Tables 5-6 and the exported image records. Source links open the original Wikimedia Commons file pages. Names are transcribed in plain Latin characters where needed.

- 001. Atcham flood road. TCExplorer. CC BY-SA 2.0. [Original file page](#).
- 002. Trebic flooded street. Jiri Sedlacek. CC BY-SA 4.0. [Original file page](#).
- 003. Oklahoma red car. Marvin Nauman/FEMA. Public domain in the US. [Original file page](#).
- 004. Oklahoma neighbourhood. Marvin Nauman/FEMA. Public domain in the US. [Original file page](#).
- 005. Flooded street and vehicles. MarkBuckawicki. CC0. [Original file page](#).
- 006. Cedar Rapids, 13th Avenue. Don Becker/USGS. Public domain in the US. [Original file page](#).
- 007. West Point flood damage. United States Military Academy. Public domain in the US. [Original file page](#).
- 008. Cedar Rapids, sunken cars. Don Becker/USGS. Public domain in the US. [Original file page](#).
- 009. Berry Street. Bidgee. CC BY-SA 2.5 Australia. [Original file page](#).
- 010. Dubai flood. CherryPie94. CC0. [Original file page](#).

Modifications: prediction boxes and labels are added only to annotated copies; figure and gallery previews are resized. Original scene images are retained in input_images. Complete filenames, authors, source links, licence identifiers and modification descriptions remain in image_credits.csv. Applicable original licence conditions must be retained when reusing or distributing image derivatives.

Experimental evidence: supplied Team20_CSE411_Project_Task7.ipynb and task7_results_20261003_181318_075801.UTC.zip. Quantitative values in this report are taken from those saved results, not from the cited research papers.